

Pistes d'optimisation — modèles IA

DeepEdgeBenchmark — ARIMA-GARCH, SARIMA, Prophet, LSTM, Naive

Etat au 21 juillet 2026

Cadrage

Ce document ne couvre que les 5 modèles « classiques » (ARIMA-GARCH, SARIMA, Prophet, LSTM, Naive) — TSDiff est volontairement exclu, aucune comparaison directe n'est faite ici. Six pistes, classées par modèle/thème concerné, chacune avec : ce qu'elle améliorerait et pourquoi, puis comment la mettre en place concrètement dans le repo.

1 Prophet

1.1 Régénérer le backtest OOS avec le correctif walk-forward déjà codé

Ce que ça améliore, et pourquoi. `run_prophet` faisait un seul *fit* puis prédisait toute la fenêtre de test en un batch, au lieu de ré-ajuster à chaque pas comme les autres modèles (`BRIEF_correction_prophet.md`) — ce qui créait une dérive de tendance mesurée à +11,8 % sur BTC et +21,8 % sur ETH. Le correctif walk-forward a été codé et mergé le 15/07, mais **jamais appliqué aux données déjà en base** : le biais mesuré aujourd'hui sur le backtest OOS de `tracking.db` reste de +10,4 %/+17 %, identique à l'avant-correctif. Tant que ce n'est pas fait, tout chiffre Prophet basé sur ce backtest quotidien (taux d'utilisation, RMSE, CRPS) reste invalide.

Comment l'implémenter. (1) Régénérer les dossiers `Run/*-Prophet*-D1/` en relançant le pipeline (`model_artifacts/pipeline.py`) — il appelle désormais la version corrigée de `run_prophet`. (2) Supprimer les lignes OOS Prophet déjà ingérées dans `predictions` (`source='oos', model='Prophet'`) — l'upsert de `insert_oos_predictions` ne les écrasera pas si le `run_id` est identique. (3) Ré-ingérer via `validation.sim_trades.ingest_oos`, qui reconstruit automatiquement `sim_trades` (`rebuild_oos_sim_trades`). (4) Vérifier que le biais retombe au même ordre que les autres modèles (§6 du brief).

1.2 Passer du prix brut au log-prix

Ce que ça améliore, et pourquoi. Indépendamment du point précédent, Prophet est fitté sur le **prix brut** — pas seulement dans `models/prophet_model.py`, mais aussi dans `benchmarks/multi_horizon.py::f` la fonction utilisée par les expériences hebdomadaires/CRPS les plus récentes (celle-là n'a jamais eu le bug walk-forward, donc le correctif du §1.1 ne la concerne pas). Sur une série non stationnaire à forte croissance comme BTC/ETH, la tendance (linéaire ou logistique) de Prophet s'extrapole de façon agressive et dérive — c'est très probablement la cause principale du CRPS catastrophique observé pour Prophet dans l'analyse probabiliste la plus récente, bien plus que le bug walk-forward. Ce point n'a, à ce jour, jamais été corrigé.

Comment l'implémenter. Dans `fit_prophet` (`multi_horizon.py`) et `run_prophet/next_step_prophet` (`models/prophet_model.py`) : remplacer `y = train.values` par `y = np.log(train.values)` au moment de construire `df_train`, puis ré-exponentier (`np.exp(...)`) le point prédit *et* les deux bornes de l'IC en sortie de `model.predict(...)`. Changement localisé (quelques lignes), sans toucher à la saisonnalité ni aux autres paramètres — à tester d'abord isolément sur BTC/ETH (les actifs les plus non stationnaires) pour confirmer la réduction de dérive avant de régénérer tous les backtests.

2 LSTM — intervalle de confiance réactif

Ce que ça améliore, et pourquoi. L'intervalle de LSTM est $\text{pred} \pm 1,96 \times \text{écart-type des résidus d'entraînement}$, calculé **une seule fois** et jamais réactualisé — contrairement au GARCH d'ARIMA-GARCH, qui adapte sa largeur aux conditions de marché courantes. Conséquence directe : la largeur de LSTM dépend presque uniquement de l'échelle historique de l'actif, pas de la volatilité récente, ce qui explique une bonne part de sa variabilité observée par actif (bande démesurée sur BTC/ETH, plus serrée sur SPY/TLT).

Comment l'implémenter. Dans `models/lstm_model.py` (`run_lstm/next_step_lstm`), remplacer le `std` figé (`np.std(train.values[seq_len:] - train_preds)`, calculé une fois sur tout le train) par une estimation glissante — soit une EWMA des carrés des résidus les plus récents (fenêtre de `N` jours, réactualisée à chaque pas du walk-forward, dans le même esprit que le GARCH), soit, plus simplement dans un premier temps, un GARCH(1,1) ajusté directement sur les résidus de LSTM (réutilisable depuis `models/arima_model.py`, qui fait déjà exactement ça pour ARIMA).

3 LSTM — entraîner sur les rendements plutôt que sur le niveau de prix

Ce que ça améliore, et pourquoi. Actuellement, LSTM apprend à prédire le **niveau de prix** scalé (MinMax), ce qui donne à l'optimiseur MSE une solution « gratuite » à faible erreur : recopier le dernier prix connu. C'est cohérent avec ce qu'on observe (mouvement moyen prédit quasi nul, très proche de la persistance pure). Entraîner sur les rendements (variations relatives d'un jour à l'autre) retire ce raccourci : la cible devient stationnaire et centrée sur 0, donc le réseau doit effectivement apprendre un signal pour faire mieux que prédire « 0 » — ce qui devrait aussi rapprocher son comportement de celui d'ARIMA-GARCH (qui, lui, modélise déjà les rendements).

Comment l'implémenter. Dans `models/lstm_model.py` : construire les séquences d'entrée/cible sur `np.diff(np.log(prices))` au lieu de `prices` scalé, garder le `MinMaxScaler` sur les rendements plutôt que sur le prix, et reconstruire le prix prédit à la sortie via `last_price * exp(cumsum(rendements prédits))`. Impact à mesurer d'abord sur le Directional Accuracy de `honest_eval` (avec IC de Wilson et p-value vs pile-ou-face) — si le gain n'est pas significatif, le changement n'apporte rien de plus qu'un raccourci différent.

4 Tous les modèles paramétriques — corriger la forme de la distribution

Ce que ça améliore, et pourquoi. L'analyse de calibration à 3 niveaux (50/80/95 %) montre un motif net : ARIMA-GARCH, SARIMA, LSTM et Naive **sur-couvrent** tous au centre (50 % visé, 0,57 à 0,66 observé) tout en étant à peu près corrects aux queues (95 %). C'est la signature classique

d’une hypothèse gaussienne/symétrique appliquée à des rendements en réalité plus resserrés au centre et à queues plus grosses qu’une gaussienne (leptokurtiques) : pour atteindre la bonne queue à 95 % avec un sigma gaussien unique, il faut le gonfler, ce qui élargit mécaniquement tout le reste, y compris le centre. Réduire l’étendue du PI *sans* changer cette hypothèse casserait la couverture aux queues — ce n’est pas un levier indépendant.

Comment l’implémenter. Remplacer la construction gaussienne symétrique ($Z_{95} = 1,96$ dans ARIMA-GARCH/LSTM/Naive, l’IC gaussien de `statsmodels` pour SARIMA) par une loi à queues épaisses — un Student-t dont le degré de liberté est estimé sur les résidus historiques (via `scipy.stats.t.fit`), en gardant le même μ/σ implicites mais en piochant les quantiles dans le Student-t plutôt que dans la loi normale — ou, plus robuste car sans hypothèse de forme, une approche empirique/bootstrap : construire l’IC directement à partir des quantiles des résidus passés (in-sample, sans fuite), plutôt que d’un z -score théorique. Les deux resserrent mécaniquement le centre tout en gardant des queues correctement dimensionnées.

5 Signal directionnel — features de momentum/volatilité

Ce que ça améliore, et pourquoi. ARIMA-GARCH, SARIMA, LSTM et Naive prédisent tous un mouvement quasi nul (persistance) — pas nécessairement un défaut à corriger en soi, mais une limite si l’objectif est d’extraire un vrai signal directionnel. Attention cependant : pousser un modèle à s’éloigner davantage de la persistance *sans* gain de compétence réel ne fait qu’importer plus d’erreur, pas plus de justesse — ce n’est un progrès que si le signal ajouté est authentique.

Comment l’implémenter. Ajouter en entrée d’ARIMA (variante ARIMAX) et de LSTM des features de momentum (rendement sur N jours) et de volatilité récente (écart-type glissant, ou sortie du GARCH existant), puis mesurer strictement l’apport via `honest_eval.metrics.directional_accuracy` (IC de Wilson, p-value vs 50 %) et `diebold_mariano` contre la version sans features — ne généraliser le changement que si le gain est statistiquement confirmé, pas simplement visible sur l’échantillon d’entraînement.

6 Mélanger les prédictions via le taux d’utilisation

Ce que ça améliore, et pourquoi. Combiner les 5 modèles en un ensemble peut réduire l’erreur individuelle de chacun (principe standard de la combinaison de prévisions). Le taux d’utilisation, calculé par modèle et par actif, est un candidat naturel de pondération — surtout vu l’écart de performance très marqué par actif observé pour LSTM.

Comment l’implémenter. Deux niveaux possibles : (a) une moyenne pondérée simple des prévisions ponctuelles, poids = taux d’utilisation normalisé par actif ; (b) plus rigoureux, un mélange des distributions complètes (gaussienne implicite de chaque modèle, pondérée) pour obtenir une distribution d’ensemble sur laquelle calculer un vrai CRPS. **Réserve importante** : le taux d’utilisation est un KPI bruité, à D+1 seulement, mesuré sur de petits échantillons (cf. discussions précédentes) — je recommande de baser les poids sur le CRPS/MASE poolé de `experiments/pooled_analysis.py` (bien plus de données, testé statistiquement), et de n’utiliser le taux d’utilisation qu’en signal secondaire, avec un *shrinkage* vers une pondération égale plutôt qu’une pondération agressive vers le modèle qui semble le meilleur sur un petit échantillon.